

INSTITUTO
FEDERAL
Santa Catarina

Resumo de matérias

Introdução à Computação

Eduardo Cardoso Oliveira, 2025

Aviso importante e Isenção de responsabilidade

1. **Finalidade do documento**
   Este documento foi elaborado com o objetivo de servir como um material de apoio e revisão para os estudos das matérias do primeiro semestre de Análise e Desenvolvimento de Sistemas. Todos os conteúdos aqui presentes foram retirados de anotações pessoais e resumos, compilados para facilitar a compreensão do leitor.

2. **Fontes e Direitos Autorais**
   Os resumos e anotações aqui presentes foram desenvolvidos a partir dos materiais de sala disponibilizados pelos professores (como slides, apostilas, listas de exercícios, livros, etc), anotações de aula e pesquisas. Os direitos autorais sobre os materiais originais (aulas, livros, artigos) permanecem integralmente aos seus respectivos autores e editoras. Este documento é uma obra derivada, de caráter estudantil e sem fins lucrativos.

3. **Precisão das Informações**
   Desde já, o autor não oferece nenhuma garantia de que os conteúdos aqui contidos sejam precisos e corretos, por mais que todo esforço tenha sido feito para garantir que tais informações sejam corretas. Erros, omissões ou interpretações equivocadas podem existir. O autor não se responsabiliza por quaisquer consequências, acadêmicas ou de outra natureza, que possam surgir do uso deste material.

4. **Uso Recomendado**
   Este material **NÃO SUBSTITUI DE FORMA ALGUMA** a leitura de bibliografias recomendadas por professores, presença em aulas, consulta às fontes oficiais e, principalmente, as orientações do corpo docente. Utilize este documento como uma ferramenta complementar aos seus estudos.

5. **Vínculo Institucional**
   Este é um projeto independente e não possui qualquer vínculo oficial com o Instituto Federal de Santa Catarina Campus São José, a Coordenação do Curso de Análise e Desenvolvimento de Sistemas, ou os professores das disciplinas. Não há qualquer endosso ou revisão oficial do conteúdo por parte da instituição.

Apresentação do Material

Olá, colega! Seja bem-vindo(a) ao meu resumo de estudos! Este documento foi criado ao longo do semestre com o objetivo de centralizar e facilitar o acesso aos conteúdos que foram abordados durante as aulas do primeiro semestre do curso em 2025.1 na disciplina de Introdução à Computação.

O que você vai encontrar neste documento?

- **Resumos:** Para esta disciplina trago resumo de slides, levantando pontos que considero os mais importantes.
- **Ferramentas:** Também trago algumas recomendações de ferramentas para utilizar aqui, que ajudam no aprendizado e organização.
- **Documentos e bibliografias:** Quando possível, vou disponibilizar materiais, resumos, listas de exercícios, gabaritos e referências bibliográficas para auxiliar.

Como este documento está organizado?

O material foi dividido por disciplinas, por ordem de andamento do semestre 2025.1. Cada sessão é um bloco de conteúdos.

Informações adicionais e contato

Este material foi feito de aluno para aluno; sugestões e colaborações são sempre bem-vindas! Se encontrar algum erro, tiver alguma sugestão para adicionar ao documento ou tirar alguma dúvida sobre qualquer matéria, fique à vontade para entrar em contato.

- **Autor:** Eduardo Cardoso Oliveira
- **E-mail Institucional:** eduardo.co22@aluno.ifsc.edu.br
- **Github:** https://github.com/EduCardoso22

Sumário

1 Introdução ao Hardware **5**
1.1 Algumas definições . . . 5
1.2 Dispositivos de entrada e saída . . . 5
1.3 Componentes do computador . . . 5

2 Soldagem **7**
2.1 Placa de circuito impresso (PCI) . . . 7
2.2 Preparação do componente . . . 7
2.3 Ferramentas . . . 7
2.4 Processo de soldagem . . . 7
2.5 Defeitos comuns . . . 8

3 Software Livre **9**
3.1 Software Proprietário . . . 9
3.2 Software Pirata . . . 9
3.3 Software Livre . . . 9
3.4 As Quatro Liberdades do Software Livre . . . 9
3.5 Exemplos de Uso de Software Livre . . . 9
3.6 Erros Frequentes . . . 10
3.7 É possível ganhar dinheiro com software livre? . . . 10

4 Linux em Modo Texto **11**
4.1 Vantagens do Modo Texto no Linux . . . 11
4.2 O Shell . . . 11
4.3 Extensão de Arquivos . . . 11
4.4 Arquivos Ocultos . . . 11
4.5 Recursos do Terminal . . . 11
4.6 Comandos Importantes . . . 12

5 Diretórios no Linux **14**
5.1 Diretórios especiais . . . 14
5.2 Caminhos completo e relativo . . . 14

6 Redirecionamento **14**

7 Caracteres curinga **15**

8 Controle de versão, Git e GitHub **16**
8.1 Controle de Versão (VCS) . . . 16
8.2 Git . . . 16
8.2.1 Comandos importantes . . . 16
8.3 Branches . . . 17
8.4 GitHub . . . 17

9 Robocode **18**
9.1 Anatomia e Restrições de um Robô . . . 18
9.2 Comportamentos e Eventos . . . 18
9.3 Robôs Avançados . . . 18

10 Usuários e permissionamento **19**
10.1 Donos, grupos e outros . . . 19
10.2 Tipos de Permissões . . . 19
10.3 Modo de Permissão Octal . . . 19
10.4 Comandos para Gerenciamento de Permissões . . . 19

11 Processos **20**
11.1 O que é um processo? . . . 20
11.2 Estados e Planos de Execução . . . 20
11.3 Identificação e Comunicação de Processos . . . 20
11.4 Comandos para Gerenciamento de Processos . . . 20

12 Compactadores **21**
12.1 O que são compactadores? . . . 21
12.2 Tipos de Compactação . . . 21
12.3 Programas e Extensões . . . 21
12.4 Comandos . . . 21

13 Shell Script **22**
13.1 O que é Shell Script? . . . 22
13.2 Primeiro Script . . . 22
13.3 Variáveis . . . 22
13.4 Argumentos e Variáveis Globais . . . 22
13.5 Estruturas de Dados e Controle . . . 22

14 VI e VIM **24**
14.1 O que são VI e VIM? . . . 24
14.2 Modos de Operação . . . 24
14.3 Comandos de Abertura, Salvamento e Saída . . . 24
14.4 Comandos de Movimentação . . . 24
14.5 Outros Comandos Úteis . . . 25

15 Informações adicionais **26**

16 Referências **27**

1 Introdução ao Hardware

1.1 Algumas definições

- **Hardware:** Parte física do computador. Um conjunto de componentes eletrônicos que se comunicam por barramentos.
- **Software:** Parte lógica do computador. É um conjunto de instruções e dados processados pelo hardware.
- **Bit:** Binary digit (Dígito binário) - Menor unidade de informação que pode ser armazenada ou transmitida, podendo assumir valores 0 ou 1. É utilizado devido a sua facilidade de armazenar dois valores diferentes.
- **Byte:** É um conjunto de 8 bits (também chamado de octeto). Seus valores variam de 0000.0000 (0 em decimal) até 1111.1111 (255 em decimal).
- **Prefixo binário:** Utilizado para facilitar a representação de números binários. Representa a multiplicação pela potência de 2 ou 1024.

1.2 Dispositivos de entrada e saída

- **Dispositivos de entrada:** Fornecem dados para operação em um programa. Permitem a comunicação no sentido do usuário para o computador. Alguns exemplos são: Teclado, Webcam, Mouse, Mouse com tela LCD, Tela sensível ao toque.
- **Dispositivos de saída:** Apresentam dados processados por um programa. Permitem a comunicação no sentido do computador para o usuário. Alguns exemplos são: Monitor, Mouse com tela LCD, Tela sensível ao toque.

1.3 Componentes do computador

- **O Computador:** Não é apenas uma "caixa metálica"e não é definido somente pelo modelo do processador. Deve ser montado tendo em vista a sua utilização.
- **Gabinete:** A caixa metálica que abriga as peças eletrônicas do PC. Possui espaço para a placa-mãe (padrões AT e ATX) e inclui a fonte de alimentação. Os formatos mais comuns são desktop e minitorre.
- **Placa-mãe:** Também chamada de motherboard ou MoBo. É onde os componentes são conectados, e deve ser escolhida de acordo com os componentes que serão usados. Os conectores da placa-mãe são chamados de soquetes e slots. O padrão ATX, por exemplo, soluciona problemas do padrão AT, como o pouco espaço interno e a baixa circulação de ar. Periféricos como placas de vídeo, som e rede podem vir embutidos na placa-mãe (on-board) para reduzir o preço, mas não podem ser trocados para upgrade ou em caso de defeito.
- **Processador:** Também conhecido como microprocessador, CPU (Central Processing Unit) ou UCP. É o "cérebro"do computador, responsável por realizar cálculos matemáticos e controlar outros equipamentos. Os processadores possuem encaixes próprios (soquetes).

- **Cooler:** Composto por um dissipador de calor e uma ventoinha. É utilizado para resfriar o processador, que esquenta devido às altas taxas de processamento. O ar frio é jogado no dissipador, que por sua vez, com a ajuda da pasta térmica, reduz a temperatura do processador. Existem outros modelos que utilizam líquidos para o resfriamento.
- **HD - Hard Disk:** Também chamado de disco rígido ou Winchester. Armazena informações na forma binária. Possui um funcionamento semelhante aos antigos discos de vinil e tem um acesso lento às informações.
- **Memória RAM:** Armazena dados que serão utilizados pelo processador. Tem uma alta taxa de acesso às informações. Os dados são armazenados apenas enquanto são utilizados, e uma queda de energia resulta na perda de dados. Existe uma grande variedade de padrões, que se diferenciam principalmente pela velocidade de acesso.
- **Placa de vídeo:** Responsável pela comunicação do computador com o monitor. Possui processadores e memórias RAM dedicados para o processamento e armazenamento de informações visuais. Jogos 3D modernos, por exemplo, exigem uma grande quantidade de memória e um bom processador de vídeo.
- **Placa de som:** Responsável pela comunicação do computador com as caixas de som. Assim como as placas de vídeo, possuem memória e processadores dedicados.
- **Estabilizador de tensão:** Protege o computador contra variações e ruídos da rede elétrica. Precisa ter potência suficiente para alimentar o computador.
- **No-break:** É um sistema de baterias que alimenta o computador temporariamente em caso de falta de energia. Geralmente possui um estabilizador interno. É muito utilizado em empresas para permitir que os trabalhos sejam salvos antes da queda total de energia.

2 Soldagem

2.1 Placa de circuito impresso (PCI)

A Placa de Circuito Impresso, também conhecida como Printed Circuit Board (PCB), serve como um suporte para a interligação de componentes eletrônicos. Ela é composta por trilhas de cobre condutor e ilhas, que são os pontos de conexão.

2.2 Preparação do componente

Para a preparação de um componente a ser soldado, siga os passos abaixo:

1. Lixar os terminais para remover a oxidação.
2. Desentortar os terminais e encaixar o componente perfeitamente na placa.
3. Dobrar levemente o terminal para evitar que ele saia da placa.
4. Após a soldagem, corte o componente no ponto ideal.

2.3 Ferramentas

- **Ferro de solda:** Utilizado para aquecer a junta a ser soldada. Suas pontas podem ser de cobre com aço, níquel ou ferro. Os tipos comuns incluem ponta cônica, pistola, machadinha e estação de solda.
- **Sugador:** Uma ferramenta com um tubo e um êmbolo de mola, usada para retirar a solda de componentes da placa.
- **Suporte para ferro:** Evita queimaduras na mão, na bancada ou no cabo de alimentação. Possui uma esponja vegetal umedecida para limpar a ponta do soldador.

2.4 Processo de soldagem

O processo de soldagem tem como objetivo a fixação mecânica e o contato elétrico dos componentes com as ilhas. A solda é geralmente uma liga de 63% de estanho e 37% de chumbo.

- A solda tem um fluxo desoxidante com temperatura de liquefação de 75 graus, e a liga estanho-chumbo com 183 graus.
- O processo começa aquecendo as partes (componente e ilha) a serem soldadas.
- O calor aquece o fio de solda, liberando primeiro o fluxo desoxidante e depois a liga.
- Por fim, o fio e o ferro de solda devem ser removidos do ponto de soldagem.

2.5 Defeitos comuns

- **Mau contato:** Geralmente causado por aquecimento insuficiente. Isso resulta em pouca ativação do fluxo, má molhagem, uma "liga pobre"e um intermetálico fraco e fino, levando à "Solda Fria".
- **Tempo ou temperatura excessivos:** Podem causar uma camada de solda grossa e frágil. Também pode levar a placas e trilhas danificadas, ilhas levantadas e outros estragos aos componentes por excesso de calor.
- **Solda ideal:** A faixa ideal de soldagem oferece uma boa ativação do fluxo e melhor desempenho.

3 Software Livre

3.1 Software Proprietário

Software proprietário é aquele cuja cópia, redistribuição ou modificação são proibidas pelo criador ou distribuidor. O código-fonte não é disponibilizado na compra do produto. Na verdade, o que se compra é a licença do produto, não o produto em si. Exemplos de software proprietário incluem MS Windows, MS Office, Adobe Photoshop e jogos em geral.

3.2 Software Pirata

O alto custo do software proprietário levou à aquisição de cópias ilegais, ou piratas. O uso de software pirata apresenta diversos problemas, tais como: é considerado crime, há possibilidade de alteração do software com vírus e keyloggers, não há suporte e o desenvolvedor é desvalorizado.

3.3 Software Livre

O movimento de software livre começou em 1983, iniciado por pesquisadores do MIT (Massachusetts Institute of Technology). Richard Stallman fundou o projeto GNU, um sistema operacional livre baseado no Unix (GNU significa "GNU is Not Unix"). Uma licença, a GNU General Public Licence (GNU GPL), foi criada para a distribuição do sistema GNU. Stallman mais tarde fundou a FSF (Free Software Foundation).

3.4 As Quatro Liberdades do Software Livre

A FSF define quatro liberdades que os usuários de software livre devem ter:

1. A liberdade de executar o programa, para qualquer propósito.
2. A liberdade de estudar como o programa funciona.
3. A liberdade de redistribuir cópias, inclusive vendendo-as.
4. A liberdade de modificar o programa e de liberar essas modificações.

O acesso ao código-fonte é um pré-requisito para que um software seja considerado livre, e as licenças de software livre garantem que o usuário não está cometendo nenhuma infração ao copiar ou modificar o software.

3.5 Exemplos de Uso de Software Livre

- **Caixas eletrônicos do Banco do Brasil:** Utilizam Linux, o que gerou uma economia de R$ 13 milhões em licenças.
- **Exército brasileiro:** Migrou cerca de 60 mil desktops para Linux, com a facilidade de customização sendo um fator decisivo.
- **Governo brasileiro:** Possui uma política de migração para software livre, que inclui a iniciativa Software Público Brasileiro para a construção de software livre para diversas áreas.

3.6 Erros Frequentes

- **Software livre não é o mesmo que software em domínio público:** Embora o software seja livre, o código ainda pertence a um desenvolvedor. O software em domínio público é um bem comum.
- **Software livre não é o mesmo que software com copyleft:** Um software livre sem copyleft pode se tornar não livre. Já o software livre com copyleft deve ser repassado com a mesma licença.

3.7 É possível ganhar dinheiro com software livre?

A venda de software livre não é proibida. Geralmente, a venda é feita em conjunto com serviços como suporte técnico, manuais e treinamentos, onde se compra o serviço em vez do software. Algumas formas de negócio incluem dar aulas particulares de Linux ou vender soluções que mostrem para a empresa como economizar com software livre.

4 Linux em Modo Texto

4.1 Vantagens do Modo Texto no Linux

- Simplicidade e estabilidade.
- Baixo consumo de recursos.
- É possível automatizar processos usando scripts.
- A configuração é mais direta, pois basta digitar o valor em um arquivo **.conf**.

4.2 O Shell

- O Shell é o programa interpretador de comandos do Linux.
- Ele analisa o texto que é digitado e produz um resultado.
- O texto é "case sensitive", o que significa que **cd**, **cD**, **Cd** e **CD** são comandos diferentes.

4.3 Extensão de Arquivos

- No Linux, os arquivos não precisam ter uma extensão, pois a informação sobre o tipo do arquivo está contida no cabeçalho.
- As extensões servem para dar informações ao usuário.
- O comando **file *** mostra o tipo de cada arquivo, independentemente de sua extensão.

4.4 Arquivos Ocultos

- Arquivos ocultos começam com o caractere **"."**.
- O comando **ls -l** não mostra arquivos ocultos, enquanto o comando **ls -la** os lista.

4.5 Recursos do Terminal

- **Tab completion:** completa o texto digitado após pressionar a tecla **TAB**. Se houver mais de um arquivo com o mesmo nome, um segundo **TAB** mostra todas as opções.
- **Terminais:** Ambientes semelhantes a estações de trabalho que podem ser acessados através de **CTRL+ALT+F[n]** onde **n** é de 1 a 6.
- **Login e Logout:** Para sair de um terminal, você pode usar o comando **exit**.

4.6 Comandos Importantes

- `man [comando]`: O manual do sistema. Fornece informações detalhadas sobre um comando. Para sair, use **q**.
- `sudo`: Permite executar um comando com permissões de superusuário (**root**).
- `pwd [opções]`: Lista o diretório atual.
- `history [opções]`: Lista o histórico dos comandos utilizados. A opção **-c** apaga o histórico.
- `clear`: Apaga os dados presentes na tela.
- `cd [diretorio]`: Altera o diretório atual.
- `ls [opções][diretorio]`: Lista os arquivos do diretório. A opção **-a** lista arquivos ocultos.
- `cp [opções][origem][destino]`: Copia um arquivo ou diretório. A opção **-r** copia diretórios recursivamente.
- `mv [opções][origem][destino]`: Move ou renomeia arquivos e diretórios.
- `mkdir [opções][diretório]`: Cria uma nova pasta. A opção **-p** cria diretórios dentro de outros diretórios.
- `rmdir [opções] [diretório]`: Remove um diretório. A opção **-p** remove o diretório e todos os acima dele, desde que não tenham mais nenhum arquivo dentro.
- `rm [opções][arquivo]`: Remove o arquivo. A opção **-r** remove diretórios e seus conteúdos recursivamente.
- `cat [arquivo]`: Lista o conteúdo do arquivo.
- `echo [mensagem]`: Imprime algo na tela.
- `wc -l`: Conta a quantidade das linhas de um documento.
- `less [arquivo]`: Lista o conteúdo do arquivo, com opções mais avançadas que o comando cat.
- `head [opções][arquivo]`: Lista as primeiras linhas de um arquivo. A opção **-n** especifica o número de linhas.
- `tail [opções][arquivo]`: Lista as últimas linhas do arquivo. A opção **-n** especifica o número de linhas.
- `reboot`: Reinicia o computador.
- `halt`: Desliga o computador.
- `ln [opção] [arquivo] [link]`: Cria um atalho de nome `link` para o arquivo. A opção **-s** cria uma ligação simbólica.

- `alias [apelido='comando']`: Cria um apelido para o comando. Sem parâmetros exibe a lista dos apelidos existentes.
- `unalias apelido`: Exclui o apelido criado pelo alias. A opção **-a** exclui todos os apelidos existentes.
- `grep [opções] [padrão] [arquivo(s)]`: Procura ocorrências do padrão dentro do(s) arquivo(s). Opções interessantes são o **-i** (ignorar maiúsculas e minúsculas), **-v** (pesquisa linhas que não possuem o padrão), **-w** (procura ocorrências do padrão em palavras inteiras) e **-c** (retorna o número de ocorrências).
- `find [diretório] [opção] [expressão]`: Procura arquivos no diretório.

5 Diretórios no Linux

A estrutura de diretórios do Linux é um pouco diferente da estrutura do Windows. Ela segue o padrão **HS - Filesystem Hierarchy Standard**. Possui uma única estrutura hierárquica para todos os arquivos.

5.1 Diretórios especiais

- `Diretório raiz (/):` É o diretório principal do sistema, onde todos os outros diretórios estão localizados.
- `Diretório atual (.):` É o diretório em que nos encontramos atualmente.
- `Diretório Home ( ):` É o diretório pessoal do usuário.
- `Diretório superior (..):` É o diretório acima do diretório atual na árvore.
- `Diretório anterior (-):` É o diretório que foi utilizado por último.

5.2 Caminhos completo e relativo

Existem formas diferentes de se navegar em diretórios. Um caminho absoluto inicia na raiz do sistema de arquivos (/) e fornece a localização completa do item. Um caminho relativo indica a localização em relação ao diretório de trabalho atual. Por exemplo, se utilizarmos o comando **cd /home/aluno** estamos utilizando a forma **COMPLETA** de navegar até o diretório aluno, outro exemplo seria usar o **cd /**. Agora, dois exemplos de navegação relativa seriam o **cd home** e **cd aluno**. Perceba que tudo o que existe no modo texto existe no modo gráfico também!

6 Redirecionamento

Os seguintes caracteres são usados para redirecionar a entrada e a saída de um programa no shell:

- `comando > arquivo`: Redireciona a saída de um programa para um arquivo ao invés da saída padrão. Caso o arquivo já tenha conteúdo, ele será substituído.
- `comando >> arquivo`: É similar ao redirecionamento simples, mas a saída do comando é adicionada ao final do arquivo, evitando que o conteúdo seja sobrescrito.
- `comando < arquivo`: Redireciona a entrada de um programa para um arquivo em vez da entrada padrão.
- `comando << palavra`: A entrada do comando é lida até que a palavra especificada seja digitada.
- `comando1 | comando2 (PIPE)`: Envia a saída do **comando1** para a entrada do **comando2** para que o processamento continue.

7 Caracteres curinga

Os caracteres curinga são caracteres especiais, usados no modo texto para facilitar operações com arquivos e diretórios, permitindo que vários sejam especificados de uma só vez. Os tipos de curingas apresentados são:

- **O asterisco *:** Substitui qualquer quantidade de caracteres. O comando **rm teste1*** apaga todos os arquivos que começam com "teste1".
- **O ponto de interrogação ?:** Substitui um único caractere. O comando **rm teste?.txt** remove os arquivos "teste1.txt", "teste2.txt", "teste3.txt"e "teste4.txt", mas não "teste10.txt".
- **Os colchetes** [ ]: Substituem um caractere por um dos caracteres dentro do conjunto especificado.
  - [**abc**] substitui um caractere por 'a', 'b' ou 'c'.
  - [**^abc**] substitui um caractere por qualquer um que não seja 'a', 'b' ou 'c'.
  - É possível combinar os colchetes. Por exemplo, [**a-z**][**0-9**] substitui um caractere de 'a' a 'z' seguido de um de '0' a '9'.

8 Controle de versão, Git e GitHub

8.1 Controle de Versão (VCS)

Um sistema de controle de versão (VCS), como o Git, mantém um registro das alterações feitas nos arquivos ao longo do tempo. Ele permite saber quais alterações foram feitas, quando, por quem e por quê. Com um VCS, é possível reverter alterações, facilita a colaboração entre desenvolvedores e pode ser útil até mesmo para um único desenvolvedor. Múltiplas edições em diversos arquivos podem ser tratadas como uma única alteração, chamada "commit".

8.2 Git

O Git é um sistema de controle de versão distribuído (VCS) criado em 2005 por Linus Torvalds para gerenciar o desenvolvimento do kernel do Linux. Ele possui três áreas principais:

- **Git directory (.git)**: O banco de dados do projeto que armazena as mudanças e o histórico de alterações.
- **Working tree**: A área fora da pasta `.git` onde as alterações são realizadas nos arquivos.
- **Staging area (index)**: A área que contém as alterações que estão marcadas para serem incluídas no próximo commit.

Os arquivos podem ter três estados:

- **Modified**: Um arquivo que foi modificado mas ainda não foi adicionado à área de preparação.
- **Staged**: Um arquivo cujas modificações foram marcadas para o próximo commit, usando o comando `git add <arquivo>`.
- **Committed**: As modificações foram salvas no VCS com o comando `git commit`.

8.2.1 Comandos importantes

- git init: Cria ou reinicializa um repositório Git vazio no diretório atual.
- `git status`: Exibe o estado da working tree e as alterações pendentes.
- `git commit -m 'mensagem'`: Salva um snapshot das alterações com uma mensagem descritiva.
- `git log`: Mostra o histórico de commits.
- `git diff`: Mostra as alterações que foram feitas, mas ainda não foram adicionadas ao commit.
- `git checkout <arquivo>`: Desfaz as alterações em um arquivo que ainda não foi adicionado à staging area.

- `git reset HEAD <arquivo>`: Retira um arquivo da staging area, permitindo que o `checkout` desfaça a alteração.
- `git commit --amend`: Sobrescreve o último commit, útil para corrigir a mensagem ou adicionar um arquivo esquecido. Deve ser evitado em repositórios compartilhados.
- `git revert HEAD`: Cria um novo commit que reverte as alterações do commit anterior, sem apagar o histórico.

8.3 Branches

Um branch é um ponteiro para um commit específico e representa uma linha de desenvolvimento independente.

- `git branch`: Lista os branches existentes.
- `git branch <nome>`: Cria um novo branch.
- `git checkout <nome>`: Altera para o branch especificado, atualizando os arquivos para o estado daquele branch.
- `git merge <branch>`: Mescla as alterações de um branch para o branch atual.
- **Merge Conflict**: Ocorre quando a mesma linha é alterada em dois branches diferentes. O desenvolvedor precisa resolver o conflito manualmente.

8.4 GitHub

O GitHub é uma plataforma para hospedar repositórios e criar um ambiente de colaboração.

- `git clone <url>`: Traz uma cópia de um repositório remoto para o diretório atual.
- `git push`: Envia os commits locais para o repositório remoto.
- `git fetch`: Traz as alterações do repositório remoto, mas não atualiza os arquivos locais.
- `git pull`: Traz as alterações do repositório remoto e as mescla com o branch local, atualizando os arquivos.

9 Robocode

O Robocode é uma plataforma de programação em Java onde desenvolvedores constroem robôs em formato de tanques para lutar uns contra os outros em um campo de batalha virtual. Foi criado por Mathew Nelson para provar que a linguagem Java não é lenta e pode ser usada na criação de jogos. A plataforma inclui uma interface de campo de batalha e um editor de robôs simplificado, que serve como um ambiente de desenvolvimento integrado (IDE) para escrever e compilar o código dos robôs.

9.1 Anatomia e Restrições de um Robô

Um robô no Robocode é composto por três partes principais que se movem de forma independente: o veículo, o canhão e o radar. Cada robô tem uma barra de energia que diminui quando é atingido e é recuperada quando ele acerta outro robô. O canhão só pode disparar quando seu calor está em zero, e o calor gerado é proporcional à potência do disparo.

9.2 Comportamentos e Eventos

A classe `Robot` oferece métodos para controlar o comportamento do robô, como:

- **Movimentação:** Métodos como `turnRight()`, `turnLeft()`, `ahead()` e `back()` controlam o movimento do veículo. O canhão e o radar também possuem seus próprios métodos de movimento, como `turnGunRight()` e `turnRadarRight()`. Por padrão, o canhão e o radar giram com o veículo, a menos que se usem as flags `setAdjustGunForRobotTurn()` e `setAdjustRadarForRobotTurn()`.
- **Disparo:** O método `fire(double power)` dispara com a energia especificada (até 3 unidades). Quanto maior a energia, maior o dano.
- **Eventos:** O radar dispara um evento ao detectar outro robô. O desenvolvedor pode sobrescrever métodos como `onScannedRobot()`, `onHitByBullet()`, `onHitRobot()` e `onHitWall()` para personalizar o comportamento do robô em diferentes situações.

9.3 Robôs Avançados

Robôs avançados estendem a classe `AdvancedRobot`, permitindo múltiplas ações simultâneas e a definição de estratégias a cada tique do relógio. Diferente dos robôs comuns que usam métodos de bloqueio como `turnRight()`, os robôs avançados usam métodos de não bloqueio como `setTurnRight()` e executam as ações com o método `execute()`.

10 Usuários e permissionamento

10.1 Donos, grupos e outros

Em um sistema Unix, os usuários são classificados em três categorias para fins de permissões:

- **Dono**: A pessoa que criou o arquivo ou diretório. É a única que pode modificar suas permissões. O dono está atrelado a um UID (user ID), cuja identificação é armazenada em `/etc/passwd`.
- **Grupo**: Permite que vários usuários tenham acesso ao mesmo arquivo. Um usuário pode pertencer a um ou mais grupos. O grupo é identificado por um GID (group ID).
- **Outros**: Usuários que não são donos e não pertencem ao grupo do arquivo. O usuário `root` (superusuário) não possui restrições de segurança.

10.2 Tipos de Permissões

Existem três permissões básicas para cada uma das três categorias de usuários (dono, grupo e outros):

- **Leitura (`r`)**: Permite ler o conteúdo de um arquivo ou listar o conteúdo de um diretório.
- **Escrita (`w`)**: Permite escrever em um arquivo ou diretório.
- **Execução (`x`)**: Permite executar um programa (arquivo) ou acessar um diretório.

As permissões são visualizadas com o comando `ls -l`.

10.3 Modo de Permissão Octal

O esquema de permissões pode ser representado por um número de 0 a 7, onde cada dígito representa as permissões de uma categoria (dono, grupo, outros). Por exemplo, o número 7 representa leitura, escrita e execução.

10.4 Comandos para Gerenciamento de Permissões

- `chmod [opções] [permissões] [arquivo]`: Altera as permissões de um arquivo ou diretório. As permissões podem ser definidas usando o formato `[ugoa][+-=][rwx]` ou o modo octal. A opção `-R` permite alterar permissões de forma recursiva.
- `chgrp [opções] [grupo] [arquivo]`: Altera o grupo de um arquivo ou diretório. A opção `-R` também é suportada.
- `chown [opções] [dono.grupo] [arquivo]`: Altera o dono e, opcionalmente, o grupo de um arquivo ou diretório. A opção `-R` também é suportada.

11 Processos

11.1 O que é um processo?

Um processo é um programa em execução. Para executar um comando, ele deve estar no caminho de procura de arquivos (`PATH`), que pode ser alterado nos arquivos `/etc/profile` ou `~/.bash_profile`.

11.2 Estados e Planos de Execução

Os processos podem ter os seguintes estados:

- **Executável**: Pode ser executado imediatamente.
- **Dormente**: Aguarda algo para ser executado.
- **Zumbi**: O processo está "morto", mas ainda existe.
- **Parado**: O processo está congelado e não pode ser executado.

Um programa pode ser executado em primeiro plano (`foreground`), bloqueando o terminal, ou em segundo plano (`background`), liberando-o. Para isso, basta adicionar `&` ao final do comando. Um processo pode ser interrompido com `CTRL+Z` e retomado com `fg` (foreground) ou `bg` (background).

11.3 Identificação e Comunicação de Processos

Cada processo é identificado por um PID (Process Identifier), um número único. O PPID (Parent Process Identifier) identifica o processo que o criou. A comunicação entre o sistema e os processos ocorre por meio de sinais, como `STOP`, `CONT`, `TERM` e `KILL`.

11.4 Comandos para Gerenciamento de Processos

- `ps`: Exibe uma "fotografia"dos processos em execução. Opções comuns incluem `a` (todos os processos), `f` (árvore de execução) e `u` (informações do usuário e consumo de recursos).
- `top`: Coleta e atualiza as informações dos processos regularmente, mostrando o consumo de recursos em tempo real.
- `kill`: Envia sinais a um processo para controlá-lo. A sintaxe é `kill -SINAL PID`.

12 Compactadores

12.1 O que são compactadores?

Compactadores são programas que reduzem o tamanho de um ou mais arquivos substituindo caracteres repetidos. A taxa de compactação depende da redundância dos dados: quanto maior a redundância, maior a compactação. Existem diferentes esquemas de compactação para diferentes tipos de arquivos, como texto, imagens, áudio e vídeo.

12.2 Tipos de Compactação

Há dois tipos principais de compactação:

- **Sem perdas**: O conteúdo descompactado é idêntico ao original, mas a taxa de compactação é menor.
- **Com perdas**: O conteúdo descompactado é diferente do original, mas atinge altas taxas de compactação. Este tipo é comum em arquivos de áudio, imagens e vídeos.

12.3 Programas e Extensões

O material menciona programas compactadores para Windows, como Winzip e WinRar, e para Linux, como `gzip`, `bzip2`, `zip` e `tar`.

- `gzip`: O compactador padrão do GNU/Linux, com boa taxa e velocidade. A extensão do arquivo compactado é `.gz`.
- `bzip2`: Oferece maior compactação que o `gzip`. A extensão é `.bz2`.
- `zip`: É compatível com Windows. A extensão é `.zip`. Ele permite comprimir múltiplos arquivos e diretórios, adicionar comentários e proteger com senha.
- `tar`: É um arquivador, ou seja, agrupa vários arquivos em um único, mas não os comprime. Ele pode ser usado em conjunto com compactadores como `gzip` (`-z`) ou `bzip2` (`-j`).

12.4 Comandos

- `gzip [opções] [arquivo(s)]`: Compacta arquivos. A opção `-d` descompacta, `-r` compacta diretórios recursivamente, e `-1` a `-9` ajusta a taxa de compactação.
- `bzip2 [opções] [arquivos]`: Funciona de forma similar ao `gzip`, mas com maior taxa de compactação.
- `zip [opções] [destino] [origem]`: Cria um arquivo `.zip`. A opção `-r` compacta diretórios, `-e` encripta com senha, `-m` remove os arquivos de origem, e `-u` adiciona arquivos. Para descompactar, usa-se o comando `unzip`.
- `tar [opções] -f [destino] [origem]`: Cria arquivos `.tar`. A opção `-c` cria um novo arquivo, `-t` lista o conteúdo, e `-x` extrai os arquivos.

13 Shell Script

13.1 O que é Shell Script?

Um shell script é uma sequência de comandos organizados em um arquivo executável. Funciona como uma linguagem de programação interpretada, e não compilada. É interpretado por um programa do tipo Shell. É comumente utilizada para tarefas como manipulação de arquivos, instalação e execução de programas, configuração automatizada do sistema e execução de tarefas repetitivas.

13.2 Primeiro Script

Para criar um shell script, é necessário:

1. Criar um arquivo de texto.
2. Inserir a linha `#!/bin/sh` no início do arquivo, conhecida como "shebang"ou "hash-bang", que especifica o interpretador de comandos a ser usado.
3. Escrever os comandos desejados, como `echo "Olá mundo!"`.
4. Salvar o arquivo.
5. Dar permissão de execução ao arquivo com o comando `chmod a+x script.sh`.
6. Executar o script com `./script.sh`.

Comentários são feitos com o caractere `#`.

13.3 Variáveis

Shell script é uma linguagem não tipada. Os nomes de variáveis podem começar com letras ou um underscore (_) e não podem ter espaços. O valor de uma variável é obtido usando $ na frente do nome (ex: `$ VAR`), e o nome da variável é obtido com $. Saídas de comandos podem ser atribuídas a variáveis usando `variavel=$(comando)` ou `variavel=`comando``. Variáveis são removidas com o comando `unset`.

13.4 Argumentos e Variáveis Globais

- **Argumentos**: Acessados por `$1`, `$2`, `$3`, etc. O total de argumentos é obtido com `$#`.
- **Variáveis Globais**: Variáveis de ambiente que podem ser usadas por múltiplos scripts e são definidas com o comando `export`. Exemplos incluem `PATH`, `USER`, `HOME` e `PWD`.

13.5 Estruturas de Dados e Controle

- **Arrays**: Armazenam um conjunto de valores. São compatíveis com Bash, mas não com Bourne Shell.

- **Entrada de Dados**: A interação com o usuário é feita com o comando `read variavel`.
- **Expressões Aritméticas**: São realizadas com a expressão `$((...))`.
- **Estruturas Condicionais**: Utilizam `if`, `elif` e `else` para realizar testes. A estrutura `case` também é uma alternativa para seleções.
- **Laços de Repetição**: Incluem o `for` (com diversos formatos) e o `while`.

OBSERVAÇÃO: Estruturas IF, CASE, FOR e WHILE normalmente não são cobradas em provas.

14 VI e VIM

14.1 O que são VI e VIM?

O VI e o VIM são editores de texto amplamente utilizados em sistemas Linux. Eles se diferenciam de processadores de texto (como Word) por não lidarem com formatação, tabelas ou figuras. O VIM é uma versão aprimorada do VI, com recursos avançados como autocorreção, auto-indentação, seleção visual, uso de expressões regulares e sintaxe colorida.

14.2 Modos de Operação

O VI/VIM possui quatro modos de operação:

- **Modo de inserção/substituição**: As teclas digitadas são inseridas diretamente no arquivo. É identificado por `-- INSERT --` ou `-- REPLACE --`.
- **Modo de comandos**: As teclas são interpretadas como comandos. É o modo padrão ao abrir um arquivo.
- **Modo visual**: Permite selecionar texto. É identificado por `-- VISUAL --`.
- **Modo de linha de comando**: Permite executar comandos como salvar, sair, abrir novos arquivos ou procurar e substituir texto.

A navegação entre os modos é feita com a tecla `Esc`.

14.3 Comandos de Abertura, Salvamento e Saída

- `vi arquivo`: Abre o arquivo especificado.
- `vi arquivo +N`: Abre o arquivo com o cursor na linha `N`.
- `:w`: Salva o arquivo.
- `:q`: Sai do editor.
- `:wq` ou `ZZ`: Salva e sai.
- `:q!`: Sai sem salvar, forçando.

14.4 Comandos de Movimentação

No modo de comandos, é possível mover o cursor sem as setas:

- `h, j, k, l`: Movem o cursor para a esquerda, para baixo, para cima e para a direita.
- `w, b, e`: Navegam entre as palavras.
- `gg, G`: Movem para o início e o fim do arquivo.
- `0, $`: Movem para o início e o fim da linha.

14.5 Outros Comandos Úteis

- **Cópia (`yank`) e Colagem (`paste`)**: `yy` copia a linha atual. `p` cola.
- **Exclusão (`delete`)**: `dd` exclui a linha atual. `x` apaga um caractere à frente e `X` apaga um caractere atrás.
- **Busca**: Usa-se `/palavra` para procurar. Navega-se com `n` (próximo) e `N` (anterior).
- **Substituição**: No modo de linha de comando, `:s/velho/novo` substitui a primeira ocorrência na linha. `:s/velho/novo/g` substitui todas na linha. `:%s/velho/novo/g` substitui todas no arquivo.
- **Undo**: `u` desfaz a última alteração.

As configurações podem ser personalizadas no arquivo `~/.vimrc`. O comando `vimtutor` oferece um tutorial interativo.

15 Informações adicionais

Durante o semestre, o professor propõe diversos laboratórios para praticar os temas abordados em sala de aula, estes laboratórios são de grande ajuda para consolidar os conhecimentos e para se preparar para as avaliações que ocorrem durante a disciplina. Eu recomendo que façam eles e que utilizem bem as aulas para tirar dúvidas (principalmente sobre os comandos do Linux que eu, particularmente, tive um pouco mais de dificuldade com os laboratórios). Na sessão a seguir, deixo referenciados todos os materiais que o professor ofereceu durante o semestre e alguns links de referência também muito bons. Algumas coisas que foram vistas durante as aulas, como a linguagem Markdown, eu não cheguei a colocar aqui pois não foram aprofundadas e nem utilizadas na disciplina de fato, mas deixo as referências destas coisas aqui também.

16 Referências

Referências

[1] GOOGLE DRIVE. Introdução à Computação. Disponível em: https://drive.google.com/drive/folders/19M7LV0aL68Oe7h_iXuFMLT_xaqhhic3C?usp=drive_link. Acesso em: 20 ago. 2025.

[2] IFSC. Diário 2024-2. In: WIKI do Campus São José. Disponível em: https://wiki.sj.ifsc.edu.br/index.php?title=ICO786201_-_Di%C3%A1rio_2024-2. Acesso em: 20 ago. 2025.

[3] URBAN, T. *Inside the Mind of a Master Procrastinator — Tim Urban — TED*. [S. l.]: TED, 6 abr. 2016. 1 vídeo (14 min 4 s). Disponível em: https://www.youtube.com/watch?v=arj7oStGLkU. Acesso em: 20 ago. 2025.

[4] UNIVERSIDADE DE SÃO PAULO. *História da Computação*. Disponível em: https://www.ime.usp.br/~macmulti/historico/index.html. Acesso em: 20 ago. 2025.

[5] MEUPC.NET. *O melhor site para montar seu PC Gamer*. Disponível em: https://meupc.net/. Acesso em: 20 ago. 2025.

[6] GOOGLE DRIVE. [O computador]. Disponível em: https://drive.google.com/file/d/14TwPxdkuwQuujdffN650ac3ENMcNYb_C/view?usp=sharing. Acesso em: 20 ago. 2025.

[7] MAPA DA PROVA. *Questões de Informática Básica - Hardware*. Disponível em: https://www.mapadaprova.com.br/questoes/informatica-basica/hardware. Acesso em: 20 ago. 2025.

[8] GOOGLE DRIVE. [Soldagem]. Disponível em: https://drive.google.com/file/d/1Gz5KTPTRz7eiXIxaB3OjhxOurtROPpgi/view?usp=sharing. Acesso em: 20 ago. 2025.

[9] GOOGLE DRIVE. [Software Livre no Banco do Brasil]. Disponível em: https://drive.google.com/file/d/1HjuHKSVW8E5-BU8YksU4J73-vn6BdDaW/view?usp=sharing. Acesso em: 20 ago. 2025.

[10] DIOLINUX. *Como instalar o VirtualBox 6.1*. Disponível em: https://diolinux.com.br/tecnologia/como-instalar-o-virtualbox-6-1.html. Acesso em: 20 ago. 2025.

[11] ORACLE. *Installing VirtualBox on Mac*. Disponível em: https://docs.oracle.com/cd/E19957-01/821-1695/installmac/index.html. Acesso em: 20 ago. 2025.

[12] GOGONI, R. *Como criar uma máquina virtual com o VirtualBox*. Tecnoblog. Disponível em: https://tecnoblog.net/responde/como-criar-uma-maquina-virtual-virtualbox/. Acesso em: 20 ago. 2025.

[13] WIKIPEDIA. *Distribuição Linux*. Disponível em: https://pt.wikipedia.org/wiki/Distribui%C3%A7%C3%A3o_Linux. Acesso em: 20 ago. 2025.

[14] GOOGLE DRIVE. [Introdução ao Linux]. Disponível em: https://drive.google.com/file/d/1JuNq5UpWMhMgBxsFoQYV7rWnZdfVRDxF/view?usp=sharing. Acesso em: 20 ago. 2025.

[15] GOOGLE DRIVE. [Introdução ao Linux]. Disponível em: https://drive.google.com/file/d/12N_VoAChZIAH3PZ3sHlVprVxwBBtnYrk/view?usp=sharing. Acesso em: 20 ago. 2025.

[16] ALURA. *Subir um projeto existente para o github*. Disponível em: https://cursos.alura.com.br/forum/topico-subir-um-projeto-existente-para-o-github-64968. Acesso em: 20 ago. 2025.

[17] SILVA, G. M. da. *Capítulo 18. Compactadores*. In: Guia Foca. Disponível em: https://www.guiafoca.org/guiaonline/inicianteintermediario/ch18.html. Acesso em: 20 ago. 2025.

[18] MARKDOWN GUIDE. *Getting Started*. Disponível em: https://www.markdownguide.org/getting-started/. Acesso em: 20 ago. 2025.